Reto Práctico — Ingeniería de Agentes con Microsoft Agent Framework
Ejecución y extensión de workflows de agentes con .NET: patrones secuencial, concurrente y condicional (GitHub Models y Microsoft Foundry)

---

1. Prerrequisitos del alumno

Antes de iniciar el reto, el alumno debe tener instalado y verificado lo siguiente en su equipo (Windows + Visual Studio Code + Polyglot Notebooks):

#	Herramienta / recurso	Propósito	Verificación
1	**.NET SDK** (versión compatible con Microsoft Agent Framework; se recomienda la LTS más reciente)	Ejecutar los notebooks de C# y compilar el código de los agentes	`dotnet --version`
2	**Git**	Clonar el repositorio de ejemplos	`git --version`
3	**Visual Studio Code**	Editor principal	—
4	Extensión de VS Code: **C# Dev Kit** (o la extensión **C#**)	Soporte del lenguaje C#	Buscar "C#" en Extensiones
5	Extensión de VS Code: **Polyglot Notebooks**	Abrir y ejecutar los notebooks .ipynb con el kernel .NET (C#)	Al abrir un .ipynb aparece el kernel ".NET (C#)"
6	**Azure CLI (az)**	Autenticación con Azure y administración de recursos	`az --version`
7	Extensión de VS Code: **Microsoft Foundry Toolkit** (opcional, recomendada)	Explorar el proyecto, los modelos desplegados y los agentes desde el editor	Ícono de Foundry en la barra lateral
8	**Cuenta de GitHub con acceso a GitHub Models**	Proveer el modelo de lenguaje de los casos 1 a 3	Acceso al catálogo de modelos de GitHub (github.com/marketplace/models)
9	**Personal Access Token (PAT) de GitHub** con permiso para usar GitHub Models	Autenticar los notebooks de los casos 1 a 3	Token generado y guardado solo en el equipo local
10	**Suscripción activa de Azure**	Alojar los recursos de Microsoft Foundry del caso 4	`az account show`
11	**Proyecto de Microsoft Foundry** (cuenta de Foundry + proyecto)	Contexto donde se consumen el modelo y los agentes del caso 4	Endpoint del proyecto visible en el portal de Foundry

#	Herramienta / recurso	Propósito	Verificación
12	**Modelo de chat desplegado** en la cuenta de Foundry	Modelo que usan los agentes del caso 4	Nombre del deployment visible en el portal de Foundry
13	Roles de Azure del alumno: **Foundry User** en el proyecto; para crear recursos y desplegar el modelo, **Contributor** (o Foundry Account Owner) sobre el grupo de recursos o la cuenta	Crear agentes e invocar modelos; aprovisionar y desplegar si el alumno no recibe el entorno listo	Validado por el administrador del curso
14	Sesión iniciada en Azure CLI (`az login`)	Autenticación con la identidad del alumno en el caso 4	`az account show`
15	Acceso a **nuget.org** y **github.com**	Restaurar paquetes NuGet desde los notebooks y clonar el repositorio	`git clone` y restauración de paquetes exitosos

Nota: No se requieren Docker, contenedores ni herramientas de infraestructura como código para este reto. El proyecto y el modelo de Foundry se crean desde el portal de Foundry o con Azure CLI. Los paquetes de Microsoft Agent Framework pueden estar en versión preliminar; usa las versiones que indique cada notebook. Los nombres exactos de las variables de configuración aparecen en la primera celda de cada notebook y en los archivos de ejemplo del repositorio.

---

2. Especificación del reto práctico

(Formato de especificación técnica para el alumno / equipo de desarrollo de agentes)

2.1 Identificación

Campo	Valor
ID del reto	RETO-MAF-WORKFLOW-01
Nombre	Ejecución, análisis y extensión de workflows de agentes con Microsoft Agent Framework en .NET
Curso	Desarrollo de agentes con .NET y Microsoft Agent Framework con Azure y Microsoft Foundry
Plataformas objetivo	GitHub Models (casos 1 a 3) y Microsoft Foundry en Azure (caso 4)
Entorno de desarrollo	Visual Studio Code (Windows) con Polyglot Notebooks (kernel .NET C#)
Fuente de los ejemplos	https://github.com/microsoft/Agent-Framework-Samples/tree/main/07.Workflow
Herramienta de infraestructura	Ninguna (sin IaC); recursos de Foundry mediante el portal de Foundry o Azure CLI

2.2 Objetivo

Que el alumno demuestre la capacidad de: preparar su entorno de desarrollo, **ejecutar los notebooks .NET del módulo 07.Workflow** que implementan los patrones **secuencial**, **concurrente** y **condicional** con **Microsoft Agent Framework**, explicar el papel de los **executors**, los **edges** y el **workflow**, ejecutar el caso condicional contra un proyecto de **Microsoft Foundry**, y **extender un workflow** agregando un nuevo agente.

2.3 Alcance

Incluye:

- Clonado del repositorio Agent-Framework-Samples y revisión de la documentación de 07.Workflow.
- Configuración de credenciales de GitHub Models y ejecución de los casos 1 (básico), 2 (secuencial multi-paso) y 3 (concurrente).
- Preparación de un proyecto de Microsoft Foundry con un modelo de chat desplegado.
- Ejecución del caso 4 (condicional con Foundry), recorriendo la rama aprobada y la rechazada.
- Extensión del workflow del caso 1 con un tercer agente.
- Documentación de la ejecución y evidencia de resultados.

No incluye:

- Contenerización, publicación o despliegue de los agentes como hosted agents en Foundry.
- Uso de las versiones en Python de los notebooks.
- Integración de CI/CD, evaluación o trazabilidad avanzada de los agentes.

2.4 Recursos requeridos

Recurso	Tipo	Notas
Repositorio de ejemplos	GitHub: microsoft/Agent-Framework-Samples	Se clona localmente; se trabaja en la carpeta 07.Workflow
Token de GitHub (PAT)	Credencial personal	Casos 1 a 3; nunca se versiona ni se comparte
Cuenta de Foundry	Microsoft.CognitiveServices/accounts	Contiene los modelos desplegados
Proyecto de Foundry	Microsoft.CognitiveServices/accounts/projects	Su endpoint se usa en el caso 4
Deployment de modelo	Microsoft.CognitiveServices/accounts/deployments	Modelo de chat; el código referencia el nombre del deployment

Recurso	Tipo	Notas
Resource Group	Microsoft.Resources/resourceGroups	Solo si el alumno crea los recursos; si el instructor entrega el entorno, no se requiere

2.5 Flujo de trabajo (pasos funcionales)

1. **Clonar el repositorio de ejemplos** y abrir la carpeta del módulo en Visual Studio Code:

```
git clone https://github.com/microsoft/Agent-Framework-Samples.git
cd Agent-Framework-Samples/07.Workflow
code .
```

Los notebooks de .NET están en la carpeta code_samples (subcarpeta dotnet) y se nombran del 01 al 04: `01.dotnet-agent-framework-workflow-ghmodel-basic`, `02.dotnet-agent-framework-workflow-ghmodel-sequential`, `03.dotnet-agent-framework-workflow-ghmodel-concurrent` y `04.dotnet-agent-framework-workflow-aifoundry-condition`. Si la estructura del repositorio cambió, localízalos por nombre.

2. **Leer el README de 07.Workflow** e identificar los componentes centrales de la arquitectura: *executors* (unidades de procesamiento; un agente es un tipo de executor), *edges* (rutas de los mensajes, con o sin condición) y *workflow* (orquestador que ejecuta el grafo por pasos y emite eventos).

3. **Verificar las herramientas** desde la terminal de VS Code:

```
dotnet --version
git --version
az --version
az login
az account show
```

4. **Configurar las credenciales de GitHub Models (casos 1 a 3):**
   - Crear un Personal Access Token en GitHub con permiso para usar GitHub Models.
   - Configurar el token y los datos del modelo como variables de entorno o en un archivo `.env` local, según lo indique la primera celda del notebook. A modo de ejemplo (confirma los nombres en el notebook):

```
GITHUB_TOKEN=<tu-token>
GITHUB_ENDPOINT=<endpoint-de-github-models>
GITHUB_MODEL_ID=<modelo>
```

   - Verificar que el archivo `.env` esté excluido del control de versiones (`.gitignore`).

5. **Caso 1 — Workflow secuencial básico.** Abrir `01.dotnet-agent-framework-workflow-ghmodel-basic.ipynb`, seleccionar el kernel .NET (C#) y ejecutar las celdas en orden. El workflow encadena el agente **FrontDesk**, que recomienda un lugar en París, con el agente **Concierge**,

que revisa la recomendación y propone una alternativa más local. Ubicar en el código la construcción del grafo:

```
var workflow = new WorkflowBuilder(frontDeskAgent)
            .AddEdge(frontDeskAgent, reviewerAgent)
            .Build();
```

6. **Caso 2 — Workflow secuencial multi-paso.** Ejecutar `02.dotnet-agent-framework-workflow-ghmodel-sequential.ipynb`. La cadena **Sales-Agent** → **Price-Agent** → **Quote-Agent** recibe una imagen de una sala más un texto, identifica los muebles, calcula precios (económico, medio y premium) y genera una cotización en Markdown. Observar cómo la salida de cada agente se convierte en la entrada del siguiente.

7. **Caso 3 — Workflow concurrente.** Ejecutar `03.dotnet-agent-framework-workflow-ghmodel-concurrent.ipynb` con la solicitud "Plan a trip to Seattle in December". Los agentes **Researcher-Agent** y **Plan-Agent** trabajan en paralelo (fan-out) y un agregador reúne ambos resultados (fan-in). Ubicar en el código `AddFanOutEdge`, `AddFanInEdge` y `WithOutputFrom`.

8. **Preparar Microsoft Foundry (caso 4):**
    - En el portal de Foundry, ubicar o crear el proyecto y copiar el **endpoint del proyecto**.
    - Desplegar un modelo de chat compatible y anotar el **nombre del deployment**.
    - Confirmar que el alumno cuenta con el rol **Foundry User** en el proyecto.
    - Iniciar sesión para que el notebook use la identidad del alumno:

```
az login
az account show
```

    - Configurar el endpoint y el nombre del deployment en las celdas o variables que indique el notebook 04.

9. **Caso 4 — Workflow condicional con Foundry.** Ejecutar `04.dotnet-agent-framework-workflow-aifoundry-condition.ipynb`. El **Evangelist-Agent** redacta un tutorial técnico, el **ContentReviewer-Agent** lo revisa (verifica que supere las 200 palabras) y una condición decide la ruta: con respuesta "Yes" continúa al **Publisher-Agent**, que guarda el contenido en un archivo Markdown; con "No" se informa el motivo del rechazo. Ejecutarlo **dos veces**: (a) para recorrer la rama aprobada y (b) pidiendo un borrador muy corto para forzar la rama rechazada. Ubicar en el código:

```
.AddEdge(contentReviewerExecutor, publishExecutor, condition:
GetCondition(expectedResult: "Yes"))
.AddEdge(contentReviewerExecutor, sendReviewerExecutor, condition:
GetCondition(expectedResult: "No"))
```

Como la salida del modelo puede variar entre ejecuciones, repite el caso si no se obtiene la rama

esperada.

10. **Extender el workflow del Caso 1.** Agregar un tercer agente llamado **Translator** que traduzca al español la revisión final del Concierge, y encadenarlo al grafo. Tomar como base el patrón del notebook:

```
AIAgent translatorAgent =
openAIClient.GetChatClient(github_model_id).CreateAIAgent(
    name: "Translator",
    instructions: "Traduce al español el texto recibido conservando el tono.");

var workflow = new WorkflowBuilder(frontDeskAgent)
            .AddEdge(frontDeskAgent, reviewerAgent)
            .AddEdge(reviewerAgent, translatorAgent)
            .Build();
```

Ejecutar de nuevo con "I would like to go to Paris." y verificar que la salida final esté en español.

11. **Documentar y recopilar la evidencia** (capturas de pantalla o salida de las celdas) y redactar el README del alumno con los comandos ejecutados, los recursos utilizados y las respuestas a las preguntas conceptuales del criterio 10.

2.6 Criterios de aceptación

#	Criterio	Verificación
1	El entorno fue preparado y verificado	Salida de `dotnet --version`, `az account show` y kernel .NET (C#) seleccionado en el notebook
2	Las credenciales se gestionan sin exponer secretos	Token en variables de entorno o `.env` ignorado por Git; ningún secreto en el código ni en los entregables
3	Caso 1 ejecutado correctamente	La salida muestra la recomendación de FrontDesk y la revisión de Concierge
4	Caso 2 ejecutado correctamente	Cotización en Markdown generada por Quote-Agent a partir de la cadena Sales → Price → Quote
5	Caso 3 ejecutado correctamente	Salida agregada con los resultados de Researcher-Agent y Plan-Agent
6	Modelo desplegado en Foundry y configurado en el caso 4	Deployment visible en el portal o en Foundry Toolkit; endpoint y nombre del deployment configurados
7	Caso 4, rama aprobada ("Yes")	El Publisher-Agent guarda el contenido en un archivo Markdown

#	Criterio	Verificación
8	Caso 4, rama rechazada ("No")	Se muestra el motivo del rechazo y no se invoca al Publisher-Agent
9	Workflow extendido con el agente Translator	El grafo tiene tres agentes encadenados y la salida final está en español
10	El alumno explica los conceptos clave en su README	Respuestas breves a: ¿qué es un executor?, ¿qué diferencia hay entre un edge simple y uno condicional?, ¿qué hacen fan-out y fan-in?

2.7 Restricciones

- Deben utilizarse los notebooks de .NET (C#) del repositorio; no se permite sustituirlos por las versiones de Python.
- No se permite escribir tokens, llaves o endpoints sensibles directamente en el código ni subirlos a un repositorio.
- El caso 4 debe ejecutarse contra un proyecto de Microsoft Foundry, autenticando con la identidad de Azure del alumno según lo indique el notebook.
- Los casos 1 a 4 deben conservar el patrón de grafo original; la única modificación permitida es la extensión del paso 10.

2.8 Entregables

1. Carpeta o repositorio con los notebooks ejecutados (con sus salidas) y la modificación del caso 1.
2. Evidencia (capturas o salida de consola) de:
    - Verificación del entorno (versiones de herramientas y sesión de Azure).
    - Ejecución exitosa de los casos 1, 2 y 3.
    - Modelo desplegado en el proyecto de Foundry.
    - Caso 4 en la rama aprobada y en la rama rechazada.
    - Workflow extendido con el agente Translator y su salida en español.
3. Breve documento (puede ser un README.md) con los comandos ejecutados, el nombre de los recursos utilizados en Azure y las respuestas a las preguntas conceptuales.

2.9 Limpieza de recursos (obligatoria al finalizar)

Para evitar costos y exposición de credenciales después del curso, el alumno debe: revocar el token de GitHub utilizado, eliminar el archivo `.env` local y borrar los agentes que se hayan creado en el proyecto

de Foundry durante el caso 4. Si el modelo se desplegó solo para este reto, eliminar el deployment:

```
az cognitiveservices account deployment delete --name <cuenta-foundry> --
resource-group <grupo-de-recursos> --deployment-name <nombre-del-deployment>
```

Si el alumno creó un grupo de recursos exclusivo para el reto, eliminarlo por completo. No elimines recursos compartidos del curso.

```
az group delete --name <grupo-de-recursos-del-reto> --yes --no-wait
```